

Índice

1. Tema 0: Información esencial de la asignatura	3
1.1. Contenido de la asignatura	3
2. Evaluación y entregas	3
2.1. Estructura de la evaluación	3
2.2. Examen final	3
2.3. Actividades guiadas (AG)	4
2.4. Trabajo práctico	5
2.5. Artículo científico	5
2.6. Foro	5
2.7. Convocatorias y entregas tardías	5
3. Tema 1: Introducción a los algoritmos (VC1)	6
3.1. Definición de algoritmo	6
3.2. Tipos de algoritmos ★	6
3.3. Propiedades de los algoritmos	6
3.4. Partes de un algoritmo	6
3.5. Algoritmos de optimización	6
3.6. Desarrollo e implementación de algoritmos ★	7
3.7. Fases del desarrollo de algoritmos	8
3.8. Ejemplo de modelado: planificación de autobuses	8
3.9. Diseñar	9
3.10. Analizar	9
3.11. Análisis de la complejidad ★	9
4. Tema 2: Algoritmos de ordenación (VC2)	11
4.1. Definición y operaciones básicas	11
4.2. Algoritmos clásicos de ordenación	11
4.3. ★ Resumen comparativo de complejidades	13
5. Tema 3 Parte 1: Diseño de Algoritmos (VC3)	14
5.1. Técnicas voraces (Greedy Algorithms) ★	14
5.2. Algoritmos voraces sobre grafos ★	14
5.3. Técnica de vuelta atrás (Backtracking)	16
5.4. Técnica de divide y vencerás	17
5.5. Técnica de programación dinámica	17
6. Tema 3 Parte 2: Problemas tipo y Algoritmos de Búsqueda	18
6.1. Programación lineal	18
6.2. Programación lineal entera	19
6.3. Problema del Agente Viajero (TSP)	19
6.4. Resolución del Problema del Agente Viajero (TSP)	20
6.5. Problema del camino más corto	21
6.6. Problema de la Mochila (Knapsack Problem)	22
6.7. Problema de Enrutamiento de Vehículos (VRP)	22
6.8. Algoritmos de búsqueda	22
7. Descenso del Gradiente	24
7.1. Introducción a la complejidad computacional	24
7.2. Análisis de algoritmos	24
7.3. Descenso del Gradiente (AGD) ★	26
7.4. Gradiente	26

7.5.	Algoritmo de descenso del gradiente	26
7.6.	Tasa de aprendizaje	27
7.7.	Descenso del gradiente y aprendizaje supervisado	28
7.8.	Variantes según el volumen de datos	28
7.9.	Trabajo práctico: aspectos clave	29
8.	Algoritmos Heurísticos	30
8.1.	Introducción a las metaheurísticas	30
8.2.	Exactos vs Heurísticos ★	30
8.3.	Elementos de las metaheurísticas	30
8.4.	Tipos de metaheurísticas	31
8.5.	Intensificación y Diversificación	31
8.6.	Teorema del No Free Lunch	31
8.7.	Búsqueda aleatoria (AG3)	31
8.8.	Búsqueda local	31
8.9.	Problema de los óptimos locales	32
8.10.	Búsqueda Tabú	32
8.11.	Recocido simulado	33
8.12.	Multiarranque	33
8.13.	GRASP	33
8.14.	Colonia de Hormigas (Ant Colony Optimization - ACO)	33
8.15.	Enjambre de Partículas (Particle Swarm Optimization - PSO)	34
8.16.	Hibridación y Otros Métodos	35
9.	Tema 7: Algoritmos Genéticos	35
9.1.	Conceptos Fundamentales	35
9.2.	Componentes y Operadores Genéticos	36
9.3.	Esquema General del Algoritmo	38
9.4.	Parámetros Críticos	39
9.5.	Parámetros	39

1 Tema 0: Información esencial de la asignatura

1.1 Contenido de la asignatura

La asignatura aborda técnicas algorítmicas para la resolución de problemas de optimización, combinando métodos deterministas, heurísticos y metaheurísticos. Entre los principales contenidos se incluyen:

- Algoritmos de ordenación.
- Técnicas de diseño de algoritmos:
 - Algoritmos voraces.
 - Divide y vencerás.
 - Programación dinámica.
 - Vuelta atrás (backtracking).
 - Ramificación y poda.
- Algoritmos de búsqueda en grafos:
 - Búsqueda en amplitud.
 - Búsqueda en profundidad.
- Descenso del gradiente.
- Métodos heurísticos y metaheurísticos:
 - Búsqueda local.
 - Búsqueda tabú.
 - Recocido simulado (*simulated annealing*).
 - GRASP.
 - Colonia de hormigas (ACO).
 - Algoritmos genéticos (GA).

2 Evaluación y entregas

2.1 Estructura de la evaluación

- Examen final: **40 %**
- Actividades guiadas: **40 %**
- Trabajo práctico: **30 %**
- Foro / debates: **10 %**

Nota importante: Es obligatorio aprobar **examen, actividades guiadas, artículo y trabajo práctico** para superar la asignatura.

2.2 Examen final

- Tipo: test.
- Número de preguntas: 20.
- Puntuación:
 - Acierto: +0.5
 - Error: -0.165
- Duración: 1 hora.

Fechas de examen

- **Primera convocatoria:** 6 de marzo, doble franja:
 - 12:00
 - 20:00
- **Segunda convocatoria:** 17 de abril, doble franja:
 - 12:00
 - 20:00

2.3 Actividades guiadas (AG)

Las actividades guiadas consisten en el desarrollo práctico de algoritmos en Python, entregadas en formato PDF.

- AG1:
 - Se presenta el 28/1/26
 - Preparación del entorno (Google Colab).
 - Componentes de un algoritmo.
 - Modelado del problema.
 - Análisis de complejidad.
- AG2:
 - Se presenta el 28/1/26
 - Algoritmos de ordenación.
 - Algoritmos voraces.
 - Backtracking.
 - Divide y vencerás.
 - Programación dinámica.
- AG3:
 - Se presenta el 6/3/26
 - Búsqueda en amplitud y profundidad.
 - Ramificación y poda.
 - Descenso del gradiente.
- AG4:
 - Se presenta el 6/3/26
 - Búsqueda local.
 - Búsqueda tabú.
 - Recocido simulado.
 - GRASP.
 - Colonia de hormigas.
 - Algoritmos genéticos.

Formato de entrega

- Entrega obligatoria en **PDF**.
- Entrega adicional en **GitHub**.
- Nombre de los archivos:
 - nombre_apellidos_AG1.pdf
 - nombre_apellidos_AG2.pdf
 - nombre_apellidos_AG3.pdf
 - nombre_apellidos_AG4.pdf
- No se permiten archivos comprimidos (.zip).

2.4 Trabajo práctico

- Resolución de un problema real de optimización.
- Desarrollo completo del algoritmo.
- Entrega en **PDF**.
- Deben identificarse explícitamente los aspectos teóricos utilizados.
- No está permitido el uso de *solvers* ni librerías específicas de optimización.
- El uso de herramientas de IA generativa debe citarse.

2.5 Artículo científico

- Lectura y análisis de un artículo científico.
- Respuesta a preguntas sobre el contenido.
- Tema: heurísticas (pendiente de confirmación).
- Entrega obligatoria.

2.6 Foro

- Una cuestión de debate obligatoria.
- Se valoran aportaciones útiles a otros estudiantes.
- Fecha límite (1^a convocatoria): **6 de marzo a las 23:59**.

2.7 Convocatorias y entregas tardías

- Primera convocatoria:
 - Entrega fuera de plazo: calificación 0.
- Segunda convocatoria:
 - Entrega antes de publicar actas: nota máxima 5/10.
 - Entrega tras publicar actas: no presentada.

3 Tema 1: Introducción a los algoritmos (VC1)

3.1 Definición de algoritmo

Un **algoritmo** es un conjunto de reglas ordenadas y finitas que describen, mediante una sucesión de pasos claramente establecidos, cómo resolver un problema.

Nos centraremos en algoritmos aplicados a problemas de cálculo y, especialmente, a problemas de **optimización**.

3.2 Tipos de algoritmos ★

3.2.1 Según la forma de cálculo

- **Algoritmos deterministas:** para una misma entrada (instancia) siempre producen la misma salida.
- **Algoritmos probabilistas:** pueden producir soluciones diferentes para la misma entrada (instancia), aunque sus pasos estén perfectamente definidos.

3.2.2 Según el tipo de solución

- **Algoritmos exactos:** garantizan la solución óptima.
- **Algoritmos aproximados:** proporcionan soluciones cercanas al óptimo.
- **Algoritmos heurísticos:** producen buenas soluciones sin garantía de optimalidad.

3.3 Propiedades de los algoritmos

Todo algoritmo debe cumplir:

- **Finitud:** debe terminar tras un número finito de pasos.
- **Precisión:** cada paso debe estar claramente especificado, sin ambigüedad.

3.4 Partes de un algoritmo

- **Entrada:** datos iniciales sobre los que opera el algoritmo.
- **Proceso:** operaciones y cálculos realizados.
- **Salida:** resultado final obtenido.

3.5 Algoritmos de optimización

La **optimización** consiste en obtener el mejor valor posible de una función objetivo dentro de un conjunto de soluciones factibles.

3.5.1 Componentes

- **Función objetivo:** medida cuantitativa de la calidad de una solución.
- **Variables:** decisiones que influyen en la función objetivo.
- **Restricciones:** condiciones que deben cumplir las variables.

3.5.2 Optimización multiobjetivo

- Se optimizan simultáneamente varias funciones objetivo.
- Las funciones suelen estar en conflicto.
- No existe una única solución óptima.
- Se busca el **óptimo de Pareto:** conjunto de soluciones no dominadas.

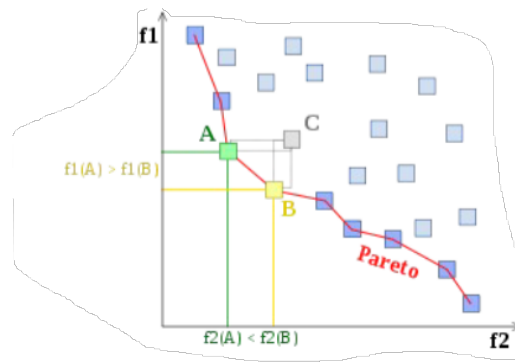


Figura 1: Óptimo de Pareto

3.6 Desarrollo e implementación de algoritmos ★

El desarrollo de un algoritmo se estructura en tres fases:

3.6.1 Modelar

Consiste en representar el problema de forma abstracta.

- Determinar supuestos y datos de partida.
- Elegir estructuras de datos adecuadas.
- Aceptar que el modelo no siempre reproduce la realidad al 100 %.

Tipos de modelos

- Icónicos.
- Analógicos.
- Simbólicos.
- Matemáticos (relaciones, ecuaciones, lógica).

Riesgos del modelado

- **Exceso:** modelo demasiado cercano a la realidad → alta complejidad.
- **Defecto:** modelo demasiado simplificado → soluciones poco realistas.

Modelos matemáticos: clasificación y resolubilidad

Según su función

- **Predictivos:** permiten estimar cómo se comportará una variable. Se apoyan habitualmente en técnicas estadísticas.
- **Evaluativos:** analizan todas las posibilidades del problema (por ejemplo, árboles de decisión).

Según los datos que se conocen

- **Deterministas:** todos los datos del problema son conocidos.
- **Estocásticos:** existen datos del problema que no se conocen con certeza.

Según la influencia del tiempo

- **Estáticos:** el paso del tiempo no influye en el comportamiento de las variables.
- **Dinámicos:** el tiempo influye en el resto de variables del modelo.

Nivel de resolubilidad del modelo El grado de dificultad para resolver un modelo depende principalmente de:

- **Tamaño del problema:** número de elementos implicados (variables, condiciones, restricciones, etc.).
- **Clase de problema:**
 - Lineal.
 - No lineal.
 - Entero (y/o binario).
 - Continuo.

3.7 Fases del desarrollo de algoritmos

El desarrollo e implementación de un algoritmo se estructura en las siguientes fases:

3.7.1 Conceptualización

- Comprender el problema a resolver.
- Representar la realidad sin contradicciones ni errores.

3.7.2 Formalización

- Establecer matemáticamente las relaciones entre los elementos del problema.

3.7.3 Evaluación

- Definir el procedimiento que permitirá resolver el problema modelado.

3.8 Ejemplo de modelado: planificación de autobuses

3.8.1 Enunciado del problema

Una ciudad desea introducir un sistema de transporte urbano mediante autobuses. Se pide estudiar el **número mínimo de autobuses** necesarios para cubrir la demanda.

- Horizonte temporal: 24 horas.
- División en franjas horarias de 4 horas.
- Restricción normativa: cada autobús puede circular **8 horas como máximo y de forma consecutiva** en cada jornada.

3.8.2 Conceptualización

- Comprender la función a optimizar.
- Identificar las restricciones del problema.
- Establecer la relación entre autobuses y franjas horarias.

3.8.3 Formalización

Definición de variables Sea X_j el número de autobuses asignados al turno j , con $j \in \{1, 2, 3, 4, 5, 6\}$, donde cada turno comienza:

- $j = 1$: 0:00
- $j = 2$: 4:00
- $j = 3$: 8:00
- $j = 4$: 12:00
- $j = 5$: 16:00
- $j = 6$: 20:00

Restricciones Cada franja horaria de 4 horas queda cubierta por dos turnos consecutivos de 8 horas, lo que da lugar a las siguientes restricciones:

$$\begin{aligned}X_1 + X_2 &\geq 8 \\X_2 + X_3 &\geq 10 \\X_3 + X_4 &\geq 7 \\X_4 + X_5 &\geq 12 \\X_5 + X_6 &\geq 4 \\X_1 + X_6 &\geq 4 \\X_1, X_2, X_3, X_4, X_5, X_6 &\in \mathbb{Z}\end{aligned}$$

Función objetivo Minimizar el número total de autobuses:

$$\text{mín } X_1 + X_2 + X_3 + X_4 + X_5 + X_6$$

3.8.4 Evaluación

Una vez definido el modelo, se debe elegir el procedimiento adecuado para resolver el problema en función del tipo de variables y restricciones.

3.9 Diseñar

- Parte creativa del proceso de resolución.
- Uso de técnicas y heurísticas que han demostrado ser útiles en la práctica.
- Demostración formal de la corrección del algoritmo para todas las posibles entradas.

3.10 Analizar

3.10.1 Aspectos a analizar

- Eficiencia temporal.
- Eficiencia en espacio.
- Comparación con otros algoritmos.

3.10.2 Dependencia del tiempo de ejecución

El tiempo de ejecución de un algoritmo depende de:

- Tamaño de la entrada n .
- Calidad de la implementación.
- Procesador utilizado.
- Complejidad del algoritmo.

3.11 Análisis de la complejidad ★

3.11.1 ★ Órdenes de complejidad

De menor a mayor crecimiento:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

3.11.2 ★ Ejemplo de cálculo

Sean n y m los tamaños de dos listas. El número de operaciones elementales es:

$$2 + n + mn + mn = 2mn + n + 2$$

Por tanto, si $m \approx n$:

$$T(n) = O(n^2)$$

Puede ocurrir que sea preferible un algoritmo peor para tamaños de entrada pequeños:

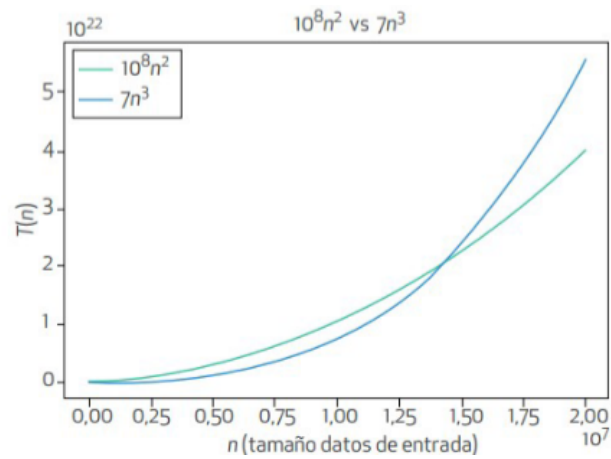


Figura 2: Diferencia computacional

4 Tema 2: Algoritmos de ordenación (VC2)

4.1 Definición y operaciones básicas

4.1.1 Definición

Ordenar una lista de valores numéricos (o, en general, una colección comparable).

4.1.2 Operaciones no elementales que se asumen

- Obtener el máximo o el mínimo de una lista: $O(n)$.
- Intercambiar dos valores de una lista: $O(1)$.

4.2 Algoritmos clásicos de ordenación

4.2.1 Inserción (Insertion Sort)

- Idea: recorrer la lista e ir construyendo una sublista ordenada insertando cada elemento en su posición.
- Adecuado para listas pequeñas y/o parcialmente ordenadas (método “manual”).
- Complejidad (depende de la disposición inicial):
 - Mejor caso: $O(n)$.
 - Peor caso: $O(n^2)$.



Figura 3: Ordenación por inserción

4.2.2 Burbuja (Bubble Sort)

- Idea: recorrer la lista intercambiando cada elemento con su adyacente si es necesario, reduciendo progresivamente el tamaño efectivo.
- Se puede mejorar deteniendo el proceso si no hay intercambios.
- Complejidad: típicamente $O(n^2)$ (independiente de la disposición inicial, salvo mejoras).

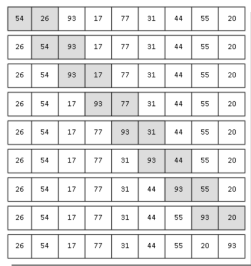


Figura 4: Ordenación por burbuja

4.2.3 Selección (Selection Sort)

- Idea: en cada iteración se busca el máximo (o el mínimo) y se coloca en su posición final, reduciendo el problema en una unidad.
- También es un método usado manualmente.
- Complejidad: $O(n^2)$ (independiente de la disposición inicial).

54	26	93	17	77	31	44	55	20	
26	54	93	17	77	31	44	55	20	
26	54	93	17	77	31	44	55	20	
26	54	17	93	77	31	44	55	20	
26	54	17	77	93	31	44	55	20	
26	54	17	77	31	93	44	55	20	
26	54	17	77	31	44	93	55	20	
26	54	17	77	31	44	55	93	20	
26	54	17	77	31	44	55	20	93	
26	54	17	77	31	44	55	20	93	

Figura 5: Diferencia computacional

4.2.4 Mezcla (Merge Sort)

- Idea: dividir recursivamente la lista en dos mitades, ordenar cada mitad y combinar (mezclar) los resultados.
- Basado en *Divide y Vencerás*.
- Se adapta bien al acceso secuencial.
- Complejidad: $O(n \log n)$ en todos los casos (se expresa típicamente mediante recurrencias).

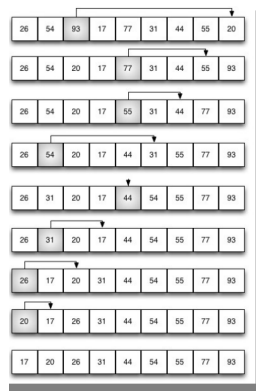


Figura 6: Ordenación por mezcla

4.2.5 Montículos (Heap Sort)

- Idea: construir un árbol binario completo (*heap*) donde el valor del padre es menor que el de sus hijos (min-heap) o mayor (max-heap).
- En cada iteración se reajusta recursivamente el árbol tras extraer/colocar elementos.
- Complejidad: $O(n \log n)$ (en el peor caso; mejora en casos favorables).
- Buen comportamiento en casos desfavorables y favorables; sin coste de almacenamiento extra significativo.
- Nota: implementación relativamente más compleja.

4.2.6 Ordenación rápida (Quick Sort)



- Idea (proceso en dos partes):
 1. Dividir recursivamente la lista en dos de forma que los elementos de la primera sean menores que los de la segunda (particionado).
 2. Combinar resultados (concatenación de sublistas ordenadas).
- Basado en *Divide y Vencerás*.
- Complejidad:
 - Peor caso: $O(n^2)$.
 - Mejor caso / promedio: $O(n \log n)$.
- Los casos favorables dependen críticamente de la elección del pivote (no de los datos en sí, sino de cómo se divide).

4.2.7 Radix Sort

- Idea: ordenaciones parciales por dígitos (de más a menos significativos o al contrario).
- Origen histórico: utilizado para clasificar tarjetas perforadoras.
- Complejidad: $O(n \cdot w)$, donde n es el número de elementos y w el tamaño (número de dígitos/longitud) de los elementos.
- Adecuado para elementos alfanuméricos (cuando la representación por dígitos es natural).

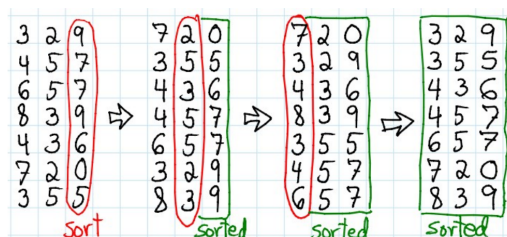


Figura 7: Ordenación por mezcla

4.3 ★ Resumen comparativo de complejidades

Algoritmo	Mejor	Medio	Peor	Espacio
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Burbuja	$O(n)$ (si se optimiza)	$O(n^2)$	$O(n^2)$	$O(1)$
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ (típico)
Radix Sort	$O(n \cdot w)$	$O(n \cdot w)$	$O(n \cdot w)$	$O(n + w)$ (depende de implementación)

Para ordenar listas, el mejor orden es el de $O(n \log n)$

5 Tema 3 Parte 1: Diseño de Algoritmos (VC3)

5.1 Técnicas voraces (Greedy Algorithms) ★

5.1.1 ★ Definición

La técnica voraz se basa en dividir el problema en etapas y, en cada una de ellas, elegir la decisión más adecuada.

5.1.2 Características de los problemas aplicables

Un problema es candidato a resolverse mediante una técnica voraz si presenta:

- Un **conjunto de candidatos** seleccionables por etapas.
- Una **solución parcial** que se va construyendo progresivamente.
- Una **función de selección** que determina el mejor candidato en cada etapa.
- Una **función objetivo** a optimizar.
- Una **función de factibilidad** que indica si una solución parcial es prometedora.
- Un **criterio de solución final** que determina cuándo una solución parcial es completa.

5.1.3 Observaciones importantes

- Es una técnica muy eficiente cuando las mejores decisiones parciales conducen a la solución óptima.
- No siempre es válida: debe demostrarse formalmente su corrección.

5.1.4 Esquema general

- Inicializar la solución vacía.
- Mientras la solución no sea completa y existan candidatos:
 - Seleccionar el mejor candidato.
 - Comprobar si es factible.
 - Incorporarlo a la solución parcial.
- Devolver la solución si es válida.

5.1.5 Ejemplo: cambio de monedas

- Objetivo: minimizar el número de monedas para una cantidad dada.
- Selección voraz: tomar tantas monedas como sea posible del mayor valor sin superar la cantidad.

Condiciones para que la técnica sea eficiente:

- Debe existir moneda de valor unitario.
- Se dispone de monedas infinitas.
- No todos los sistemas monetarios admiten solución voraz óptima (ej.: $\{1, 5, 11\}$ para $C = 15$).

5.2 Algoritmos voraces sobre grafos ★

5.2.1 Conceptos básicos ★

- Un **grafo** es un par (V, A) donde V son vértices y A aristas.
- Puede ser dirigido o no dirigido (flechas).
- Puede ser ponderado si las aristas tienen peso.

- Un **árbol** conecta todos los nodos sin ciclos.

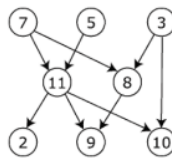


Figura 8: Grafo

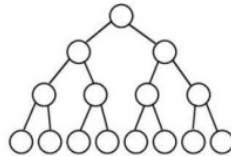


Figura 9: Árbol

5.2.2 Árbol de recubrimiento mínimo

Es un árbol de un grafo ponderado que conecta todos los vértices con **peso total mínimo**.

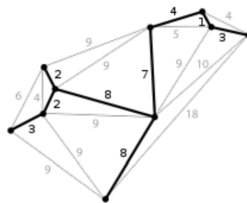


Figura 10: Árbol de recubrimiento mínimo

5.2.3 Algoritmos principales

Algoritmo de Prim

- Parte de un vértice inicial.
- En cada paso selecciona la arista de menor coste que conecta el conjunto seleccionado con el no seleccionado.

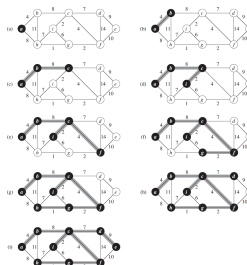


Figura 11: Algoritmo de Prim

Algoritmo de Kruskal

- Ordena las aristas por peso creciente.
- Selecciona aristas evitando ciclos.

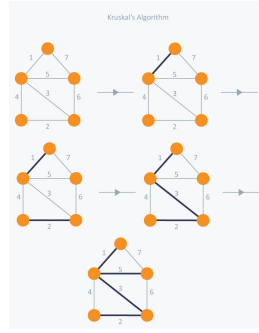


Figura 12: Algoritmo de Kruskal

5.2.4 Complejidad

- Prim: $O(n^2)$.
- Kruskal: $O(a \log n)$, donde a es el número de aristas y n el de vértices.

La elección depende de la densidad del grafo.

5.3 Técnica de vuelta atrás (Backtracking)

5.3.1 ★ Definición

Consiste en construir sistemáticamente todas las soluciones posibles por etapas, representadas como una tupla $(x_1, x_2, \dots, x_n)$, explorando un árbol de expansión.

5.3.2 Características

- Problemas discretos.
- Soluciones representables mediante un árbol.
- Posibilidad de descartar ramas (ramificación y poda).

5.3.3 Observaciones

- La complejidad suele ser exponencial.
- Identificar restricciones implícitas mejora notablemente la eficiencia.

5.3.4 Ejemplo: problema de las n reinas (AG1)

- Representación mediante n -tuplas.
- Restricciones:
 - No dos reinas en la misma fila.
 - No dos reinas en la misma columna.
 - No dos reinas en la misma diagonal: $|x_i - x_j| = |i - j|$.

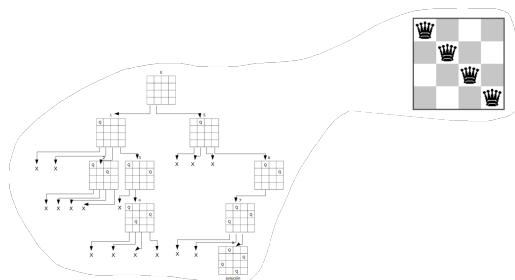


Figura 13: Problema 4 reinas

5.4 Técnica de divide y vencerás

5.4.1 ★ Definición

El problema se divide recursivamente en subproblemas más pequeños de la misma naturaleza, cuyas soluciones se combinan para obtener la solución final. Forma parte del ámbito de la algoritmia, divide y vencerás != recursividad

5.4.2 Características

- Subproblemas disjuntos.
- Resolución recursiva.
- Combinación de soluciones parciales.
- Terminación garantizada.

5.4.3 Ejemplo: Torres de Hanói

- Mover n discos de una torre a otra usando una torre auxiliar.
- Restricciones:
 - Mover un disco cada vez.
 - No colocar un disco grande sobre uno pequeño.
- Estrategia:
 - Resolver el problema para $n - 1$ discos.
 - Mover el disco mayor.
 - Resolver de nuevo para $n - 1$ discos.

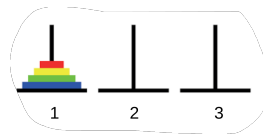


Figura 14: Problema Torres de Hanói

5.4.4 Recursividad

Forma parte del ámbito de la computación

- Facilita diseños claros.
- Incrementa el uso de memoria.
- La complejidad se analiza mediante ecuaciones en recurrencia.

5.5 Técnica de programación dinámica

5.5.1 ★ Definición

Consiste en dividir el problema en subproblemas y **almacenar** las soluciones parciales para reutilizarlas posteriormente.

5.5.2 Características

- Almacenamiento de soluciones intermedias.
- Cumple el principio de optimalidad de Bellman.
- Requiere atención a la complejidad espacial.

5.5.3 Ejemplo: viaje por el río (AG1)

- Objetivo: encontrar la ruta más barata entre embarcaderos.
- Restricción: solo se permite desplazarse río abajo.
- Se utilizan tablas de costes y soluciones parciales.



Figura 15: Problema viaje por el río

5.5.4 Relación de recurrencia

$$C(i, j) = \min \{T(i, j), T(i, k) + C(k, j) \quad \forall i < k \leq j\}$$

6 Tema 3 Parte 2: Problemas tipo y Algoritmos de Búsqueda

6.1 Programación lineal

6.1.1 ★ Definición

Son problemas de optimización en los que se desea maximizar o minimizar una función objetivo **lineal**, continua y multivariable, sujeta a un conjunto de restricciones expresadas mediante igualdades o desigualdades también lineales.

- La función objetivo es una expresión algebraica donde todas las variables son de grado 1.
- Existen numerosos problemas reales formulables como programación lineal: flujo, inventario, asignación de recursos, planificación, gestión de carteras, etc.

6.1.2 Resolución

- **Método del Simplex** (Dantzig, 1949).
- **Métodos del Punto Interior**.

Dependiendo de la región factible delimitada por las restricciones, el problema puede tener:

- Una única solución.
- Infinitas soluciones.
- Solución no acotada.
- Ninguna solución.

6.1.3 Método del Simplex

- Obtiene soluciones iterativamente desplazándose por la frontera de la región factible.
- No es necesario evaluar todos los vértices extremos.
- Sigue siendo el método más utilizado en la práctica.
- Está integrado como funcionalidad en herramientas como Excel.

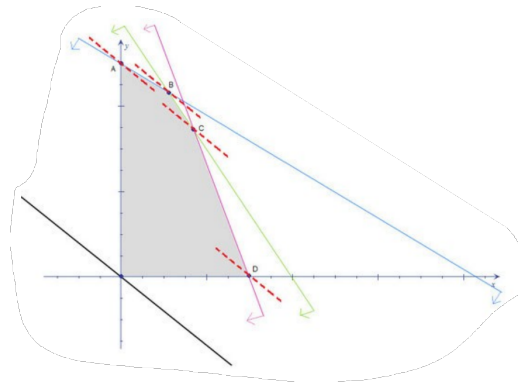


Figura 16: Método del Simplex

6.2 Programación lineal entera

6.2.1 Definición

Son problemas de programación lineal en los que se exige que algunas o todas las variables tomen valores enteros.

- **Puros:** todas las variables son enteras.
- **Mixtos:** solo algunas variables son enteras.

6.2.2 Resolución

- Resolver primero el problema relajado (sin restricción de integridad).
- Si la solución es entera, se acepta directamente.
- El redondeo de soluciones **no es válido**.
- Técnicas habituales:
 - Ramificación y poda.
 - Algoritmos heurísticos.

6.2.3 Ejemplos típicos

- Problema de inversión de capital (selección de cartera).
- Problema de coste fijo.
- Problemas de cubrimiento de conjuntos.

6.3 Problema del Agente Viajero (TSP)

6.3.1 Definición

El problema del agente viajero consiste en encontrar el recorrido de coste mínimo que permite visitar todas las ciudades exactamente una vez, comenzando y terminando en la misma ciudad.

- Se modela mediante un grafo ponderado.
- Los nodos representan ciudades.
- Las aristas representan distancias o costes.

6.3.2 Características

- Es uno de los problemas más estudiados en optimización.
- No existe algoritmo polinómico conocido que lo resuelva.
- Es un problema **NP-difícil** (NP-hard / NP-completo).

6.4 Resolución del Problema del Agente Viajero (TSP)

6.4.1 Formulación como problema de optimización

Se modela el TSP mediante variables binarias que indican si se viaja directamente de una ciudad a otra.

Variables de decisión

$$x_{ij} = \begin{cases} 1 & \text{si se visita la ciudad } j \text{ después de la } i, \\ 0 & \text{en otro caso.} \end{cases}$$

Parámetros

- c_{ij} : coste (o distancia) de ir de la ciudad i a la ciudad j .

Función objetivo Minimizar el coste total del recorrido:

$$\min \sum_{(i,j)} c_{ij} x_{ij}$$

6.4.2 Restricciones básicas

Salida única desde cada ciudad Desde cada ciudad se debe salir exactamente una vez:

$$\sum_{j=1}^n x_{ij} = 1 \quad \forall i$$

Entrada única a cada ciudad A cada ciudad se debe llegar exactamente una vez:

$$\sum_{i=1}^n x_{ij} = 1 \quad \forall j$$

Estas restricciones garantizan que cada ciudad se visita una única vez, pero **no impiden la aparición de subciclos**.

6.4.3 Eliminación de sub-tours

Restricciones clásicas de eliminación de subciclos Para cualquier subconjunto de ciudades $S \subset \{1, \dots, n\}$:

$$\sum_{i \in S} \sum_{j \in S} x_{ij} \leq |S| - 1 \quad \forall S \subset \{2, \dots, n\}$$

- Evitan la aparición de ciclos parciales.
- Número de restricciones: aproximadamente 2^n .
- Impracticables para valores grandes de n .

Conclusión Aunque son correctas desde el punto de vista teórico, estas restricciones no son viables computacionalmente para instancias grandes.

6.4.4 Formulación de Miller–Tucker–Zemlin (MTZ)

Para reducir el número de restricciones, se introduce una formulación alternativa.

Variables auxiliares

$$u_i \in \mathbb{R} \quad \forall i$$

Restricciones MTZ

$$u_i - u_j + nx_{ij} \leq n - 1 \quad \forall i \neq j$$

con:

$$x_{ij} \in \{0, 1\}$$

- Eliminan subciclos de forma implícita.
- Número de restricciones: aproximadamente n^2 .
- Mucho más manejable que la formulación clásica.

6.4.5 Complejidad del TSP

- Es un problema **NP-difícil**.
- No se conoce ningún algoritmo polinómico que lo resuelva en general.

Para un grafo de n ciudades, el número de rutas posibles es:

$$\frac{(n-1)!}{2}$$

6.4.6 Crecimiento del espacio de soluciones

Ciudades	Rutas posibles
4	3
5	12
6	60
7	360
8	2 520
9	20 160
10	181 440
20	$\approx 6 \times 10^{16}$

6.4.7 Consecuencia práctica

- La fuerza bruta es inviable incluso para tamaños moderados.
- Para instancias grandes es necesario recurrir a:
 - Ramificación y poda.
 - Heurísticas.
 - Metaheurísticas.

6.5 Problema del camino más corto

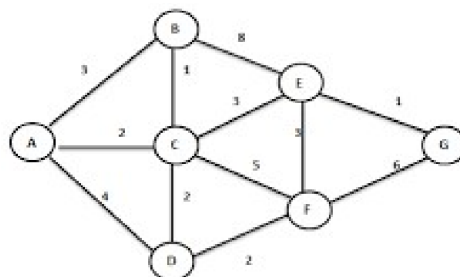


Figura 17: Problema del camino más corto

6.5.1 Definición

Consiste en encontrar el camino de menor coste entre dos nodos de un grafo, dirigido o no dirigido.

- No debe confundirse con el TSP.
- Puede requerir encontrar uno o todos los caminos mínimos.

6.5.2 Algoritmos principales

No son importantes ahora, nosotros los resolveremos con métodos heurísticos, pero estos métodos no son siempre adecuados, por lo que es importante saber que existen los siguientes:

- **Dijkstra**: eficiente, no admite costes negativos.
- **Bellman-Ford**: admite costes negativos, más lento.
- **Floyd-Warshall**: caminos mínimos entre todos los pares.
- **A***: búsqueda informada mediante heurísticas.

6.6 Problema de la Mochila (Knapsack Problem)

6.6.1 Definición

Seleccionar un subconjunto de objetos con peso y valor asociados, de forma que se maximice el valor total sin superar una capacidad máxima.

6.6.2 Resolución

- Programación lineal entera.
- Programación dinámica.
- Vuelta atrás.
- Métodos heurísticos.

6.7 Problema de Enrutamiento de Vehículos (VRP)

6.7.1 Definición

Variante del TSP en la que intervienen múltiples agentes (vehículos).

- Se pueden añadir restricciones adicionales:
 - Ventanas temporales.
 - Orígenes o destinos obligatorios.
- Es un problema complejo y de investigación activa.

6.8 Algoritmos de búsqueda

6.8.1 Definición

Son técnicas destinadas a localizar uno o varios elementos con ciertas propiedades dentro de una estructura de datos, habitualmente árboles o grafos.

6.8.2 Búsqueda en amplitud (BFS) o anchura

- Explora todos los nodos de un nivel antes de pasar al siguiente.
- Se implementa mediante una cola.

Búsqueda A*

- Variante de BFS que incorpora heurísticas. Una heurística es una regla o función que nos limita el espacio de soluciones.
- Función de evaluación:

$$f(n) = g(n) + h(n)$$

6.8.3 Búsqueda en profundidad (DFS)

- Explora primero los nodos más profundos.
- Se implementa priorizando los nodos hijos.

Una

6.8.4 Ramificación y poda

- Mejor que Vuelta atrás
- Se eliminan ramas que no pueden mejorar la solución.
- Uso de cotas inferiores y superiores.

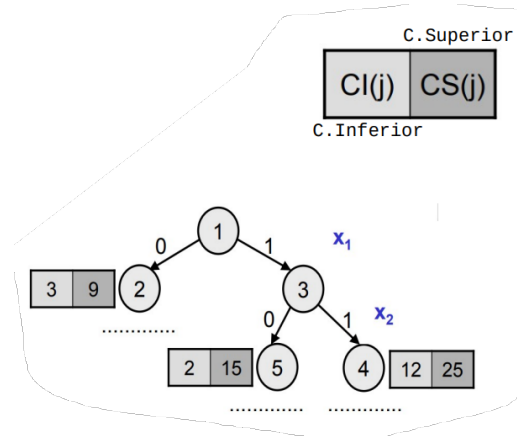


Figura 18: Problema ramificación y poda

El nodo 2 no puede mejorar (valor 9) las peores predicciones de 4 (valor 12 = $\max(\text{CI})$)
SE ELIMINA NODO 2 El nodo 5 quizá pueda mejorar al nodo 4, NO SE ELIMINA

6.8.5 Minimax

- Aplicación de búsqueda en juegos con adversario.
- Se minimiza la pérdida máxima.

7 Descenso del Gradiente

7.1 Introducción a la complejidad computacional

Se revisan conceptos básicos de complejidad para contextualizar la necesidad de métodos aproximados en problemas de optimización.

- **P**: problemas resolubles en tiempo polinomial.
- **NP**: problemas cuya solución es fácil de verificar, pero no se conoce algoritmo eficiente.
- **NP-Completo**: si uno se resuelve en P, entonces $P = NP$.
- **NP-Duro**: al menos tan complejos como los NP, aunque pueden no pertenecer a NP.

Ejemplos:

- P: ordenación de una lista.
- NP: Sudoku grande.
- NP-Completo: SAT.
- NP-Duro: Problema del Agente Viajero (TSP).

Dado que se asume que $P \neq NP$, muchos problemas reales solo pueden abordarse mediante **heurísticas y métodos aproximados**.

7.2 Análisis de algoritmos

El análisis de algoritmos se basa en **contar operaciones elementales**:

- Operaciones aritméticas.
- Asignaciones.
- Comparaciones y condiciones.
- Accesos a estructuras de datos.
- Atención especial a bucles anidados.

Un algoritmo tiene complejidad $O(f(n))$ si su número de operaciones está acotado por una constante multiplicativa de $f(n)$.

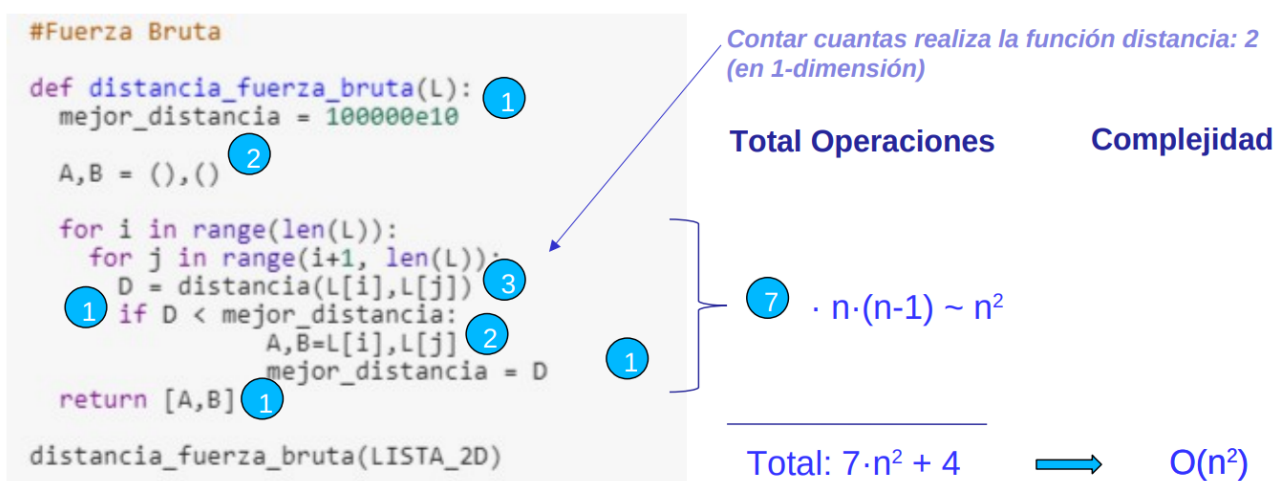


Figura 19: Cálculo orden complejidad

1 asignación 2 asignaciones Dentro del bucle 7 asignaciones: 3 asignaciones (suponemos que la función distancia tiene 2) 2 asignaciones 1 asignación return otra asignación

El termino n-simo depende de los términos anteriores

```
#Sucesión de Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Cálculo del termino n-simo de la sucesión de Fibonacci
def Fibonacci(N:int):
    if N < 2:
        return 1
    else:
        return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) - T(n-1) - T(n-2) = 0$$

Ecuación característica:

$$x^2 - x - 1 = 0$$

$$x_1 = \frac{1 + \sqrt{5}}{2} \quad x_2 = \frac{1 - \sqrt{5}}{2}$$

La solución es

$$T(n) = A \cdot \left(\frac{1 + \sqrt{5}}{2} \right)^n + B \cdot \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

A y B dependen de los términos iniciales

Figura 20: Cálculo orden complejidad

A=1, B=1, los dos primeros términos iniciales En una operación en recurrencia cuando los términos dependen de los predecesores, el cálculo del orden de complejidad se basa en realizar una expresión de segundo grado, como en la función de Fibonacci.

El termino n-simo depende de una división de los términos anteriores

```
#quick_sort
A = [9187, 244, 4054, 9222, 8373, 4993, 5265, 5470, 4519, 7182, 2035, 3506, 4337, 7580, 2554, 2824, 8357, 4447, 7379]

def quick_sort(A):
    if len(A) == 1:
        return A
    if len(A) == 2:
        return [min(A), max(A)]

    IZQ=[]
    DER=[]

    #pivot = (A[0]+A[1]+A[2])/3 #Falla en algunos casos, cuando hay valores repetidos y coinciden al principio de alguna lista
    pivot = sum(A)/len(A) #Buena opción. La mejor opción sería la mediana pero esto pasa por ordenar la lista
    # ¿que es precisamente lo que queremos hacer?

    for i in A:
        if i <= pivot:
            IZQ.append(i)
        else:
            DER.append(i)

    return quick_sort(IZQ) + quick_sort(DER)

@calcular_tiempo
def QS(A):
    return quick_sort(A)

print(QS(A))
```

Ecuaciones

$$T_n = \begin{cases} c \cdot n^k, 1 \leq n < b \\ a \cdot T\left(\frac{n}{b}\right) + c \cdot n^k, n \geq b \end{cases}$$

Soluciones

$$T(n) = O(n^k) \text{ si } a < b^k$$

$$T(n) = O(n^k \cdot \log(n)) \text{ si } a = b^k$$

$$T(n) = O(n^{\log_b(a)}) \text{ si } a > b^k$$

$$T(n) = 2 \cdot T(n/2) + n$$

$$a=2$$

$$b=2$$

$$c=1$$

$$k=1$$

En el mejor caso!

(*) El termino n-simo depende de una división de los términos anteriores

Figura 21: Cálculo orden complejidad

En este caso la recursividad se basa en dividir el problema en 2, encontraremos también ecuaciones para determinar el orden de complejidad

Órdenes de complejidad (REPASO) ★

De menor a mayor crecimiento:

$$O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3), O(2^n), O(n!)$$

7.3 Descenso del Gradiente (AGD) ★

El **Descenso del Gradiente** es un algoritmo iterativo de optimización para encontrar **mínimos** de funciones:

- Multivariantes.
- Diferenciables.
- Convexas (para garantizar óptimo global).

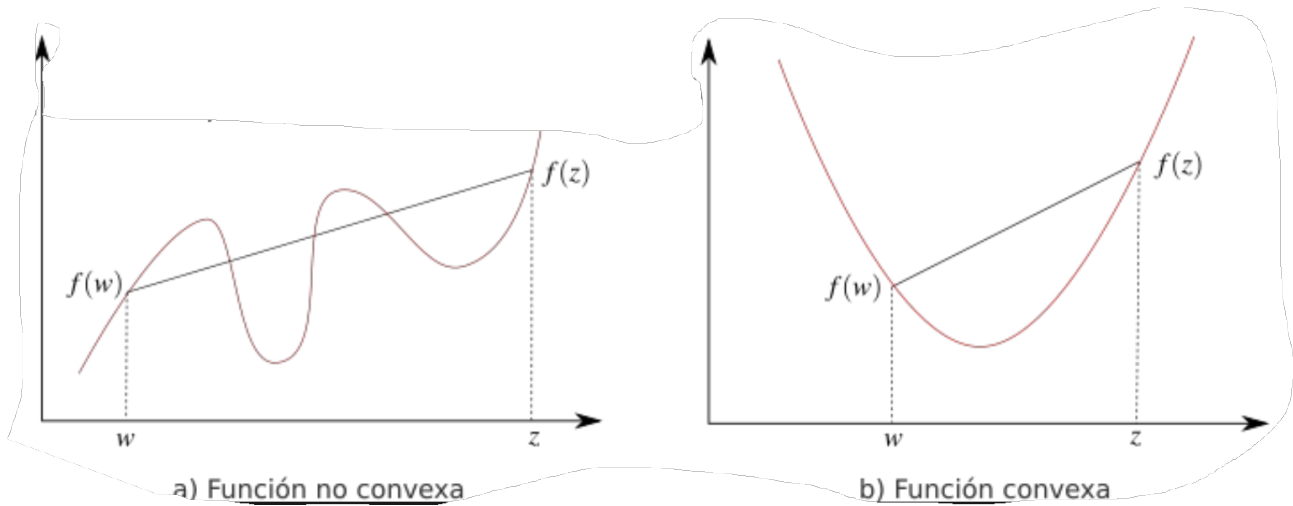


Figura 22: Convexa vs no Convexa

Convexidad ★

La convexidad permite asegurar que el mínimo encontrado es global. Si la función es cóncava y se desean mínimos, se trabaja con $-f(x)$.

7.4 Gradiente

El gradiente es un **vector de derivadas parciales**:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

Indica la dirección de máximo crecimiento de la función en un punto.

7.5 Algoritmo de descenso del gradiente

Partiendo de un punto inicial $\mathbf{p}_t$, el algoritmo avanza en sentido opuesto al gradiente:

$$\mathbf{p}_{t+1} = \mathbf{p}_t - \alpha_t \nabla f(\mathbf{p}_t)$$

La tasa de aprendizaje α_t debe garantizar:

- Soluciones factibles.
- Mejora del valor de la función objetivo.

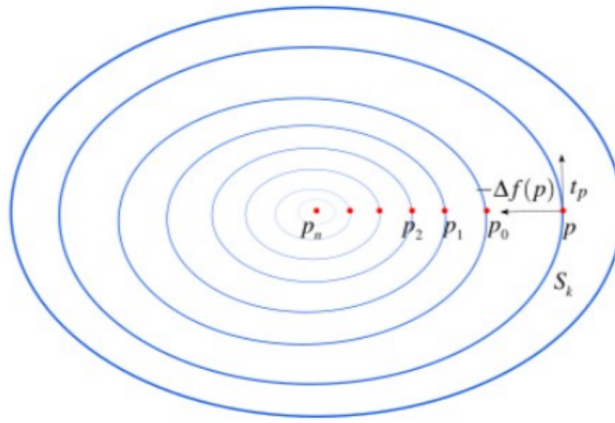


Figura 23: Acercándonos a la solución factible

7.6 Tasa de aprendizaje

La elección de α_t es crítica ★:

- α_t muy pequeño $\rightarrow$ demasiadas iteraciones.
- α_t muy grande $\rightarrow$ no converge.

Es habitual **reducir dinámicamente** α_t a medida que el algoritmo se aproxima al óptimo.
Criterios de parada:

- Magnitud del gradiente (pendientes muy pequeñas al acercarse).
- Número de iteraciones.

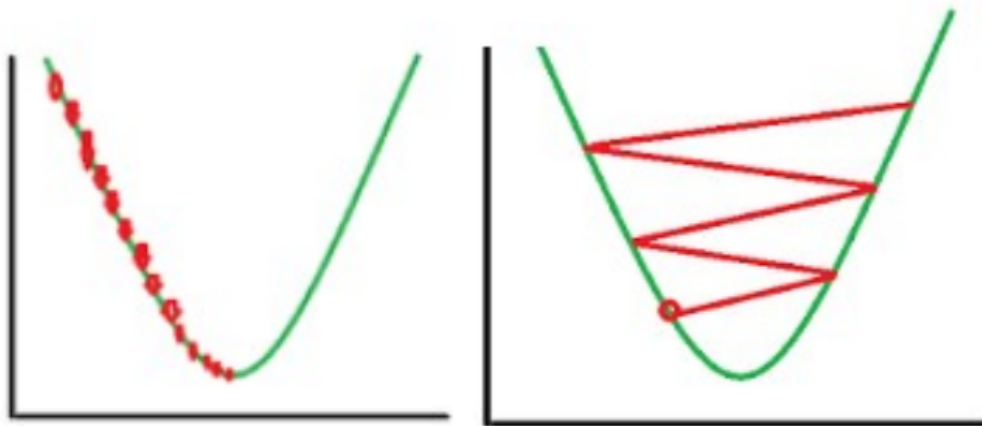


Figura 24: Diferentes tasas de aprendizaje

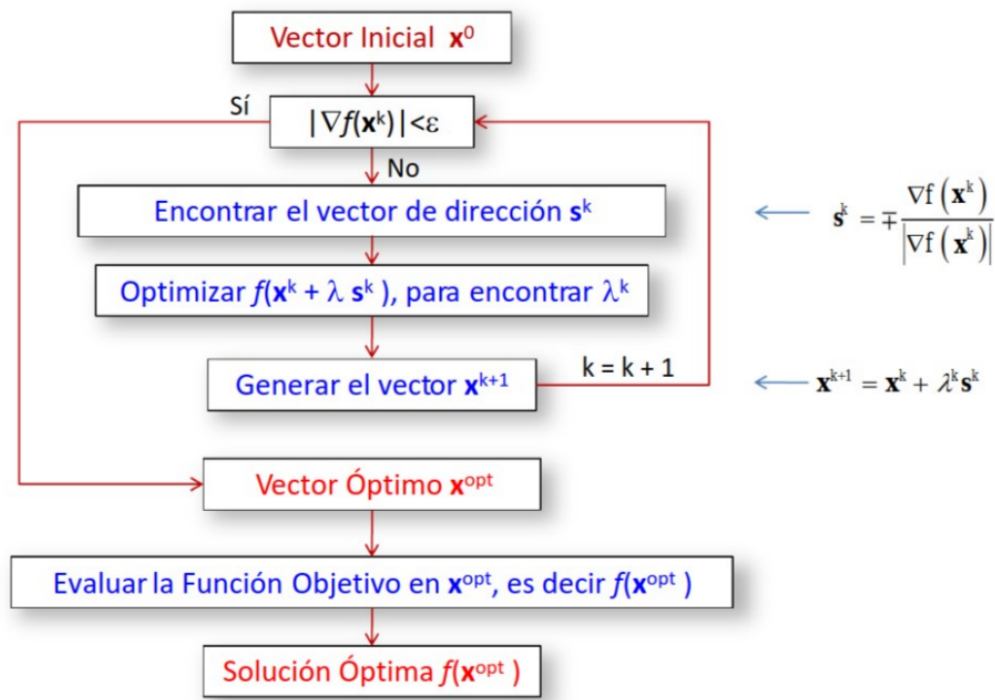


Figura 25: Proceso de resolución

7.7 Descenso del gradiente y aprendizaje supervisado

En regresión lineal:

- Se minimiza el error de clasificación.
- Se usa normalmente el **error cuadrático medio**.
- La función de coste es cuadrática y diferenciable.

7.8 Variantes según el volumen de datos

- **Batch Gradient Descent**: usa todos los datos (preciso pero lento). Calcula la desviación de todos los datos en cada iteración.
- **Stochastic Gradient Descent**: usa un solo dato por iteración (rápido pero inestable). Calcula la desviación para un punto en cada iteración.
- **Mini-batch Gradient Descent**: equilibrio entre ambos.

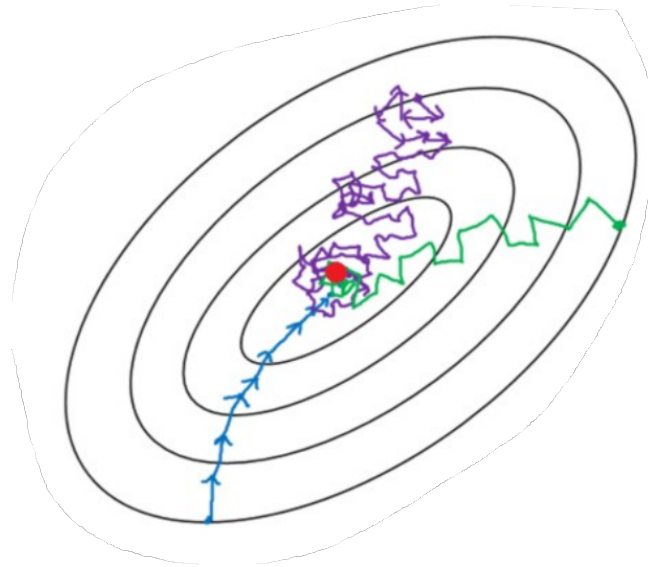


Figura 26: Variantes según volumen de datos

7.9 Trabajo práctico: aspectos clave

Además de resolver el problema, se debe justificar:

- **Modelo:** espacio de soluciones, función objetivo y restricciones.
- **Análisis:** complejidad y tamaño del espacio de búsqueda.
- **Diseño:** técnica utilizada y justificación.

Fechas límite

- Primera convocatoria: **6/03/2026**.
- Segunda convocatoria: **17/04/2026**.

8 Algoritmos Heurísticos

8.1 Introducción a las metaheurísticas

Los algoritmos heurísticos y metaheurísticos surgen como alternativa a los métodos exactos cuando los problemas de optimización son demasiado grandes o complejos.

- Permiten obtener **buenas soluciones en tiempo razonable**.
- No garantizan la optimalidad.
- Son especialmente útiles en problemas NP-difíciles.

Terminología

- **Heurístico**: algoritmo **concreto** diseñado para un problema específico.
- **Metaheurística**: estrategia **general** que guía la búsqueda de soluciones.

8.2 Exactos vs Heurísticos ★

- **Exactos / deterministas**:
 - Garantizan el óptimo.
 - Tiempo crece rápidamente con el tamaño del problema.
 - Adecuados solo para problemas pequeños.
- **Heurísticos**:
 - Obtienen soluciones de calidad, pero no exactos.
 - Tiempo de ejecución controlable.
 - Adecuados para problemas grandes.

Medidas de calidad

- **Eficacia**: esfuerzo computacional requerido.
- **Bondad**: cercanía al óptimo.
- **Robustez**: probabilidad de obtener buenas soluciones de forma consistente.

8.3 Elementos de las metaheurísticas

Las metaheurísticas combinan ideas de diferentes disciplinas:

- Técnicas de diseño de algoritmos:
 - Algoritmos voraces.
 - Divide y vencerás.
 - Programación dinámica.
- Inspiración:
 - Biología (evolución, hormigas).
 - Física (recocido).
 - Psicología (aprendizaje).
- Estadística:
 - Aleatorización.
 - Distribuciones de probabilidad.

8.4 Tipos de metaheurísticas

- **Basadas en trayectorias:**
 - Búsqueda local.
 - Búsqueda tabú.
 - Enfriamiento simulado.
- **Métodos constructivos:**
 - Multiarranque.
 - Colonia de hormigas (GRASP).
- **Basadas en poblaciones:**
 - Algoritmos genéticos.
 - Evolutivos.

8.5 Intensificación y Diversificación

Conceptos clave ★

- **Intensificación:** explorar en profundidad regiones prometedoras.
- **Diversificación:** explorar nuevas regiones del espacio de soluciones.

El objetivo de una metaheurística es encontrar un equilibrio entre ambos conceptos.

8.6 Teorema del No Free Lunch

David Wolpert y William Macready (1997) demostraron que:

- No existe un algoritmo que sea mejor que todos los demás para todos los problemas.
- Un algoritmo puede ser muy eficaz para ciertos problemas y malo para otros.

Conclusión Es necesario incorporar **conocimiento específico del problema** en el diseño del algoritmo.

8.7 Búsqueda aleatoria (AG3)

Definición Proceso que genera soluciones aleatorias y devuelve la mejor encontrada.

- Muy simple.
- Poco eficiente en espacios de búsqueda grandes.
- Sirve como referencia o método base, no como técnica final.

Si hay m soluciones posibles, la probabilidad de encontrar el óptimo en una iteración es $1/m$.

8.8 Búsqueda local

Definición Proceso iterativo que, partiendo de una solución inicial, va generando soluciones vecinas, construyendo una trayectoria en el espacio de soluciones.

- Muy orientada a la intensificación.
- Riesgo de quedar atrapada en óptimos locales.

Elementos básicos

- Solución inicial.
- Representación de las soluciones.
- Operador de vecindad.
- Criterio de parada.

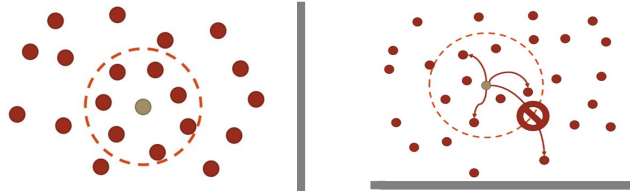


Figura 27:

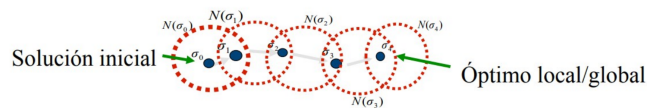


Figura 28:

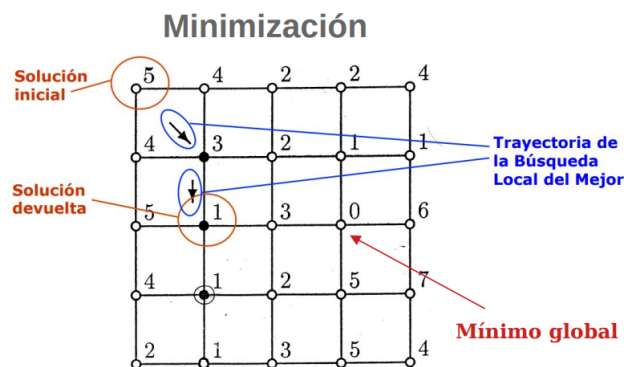


Figura 29: Desventajas

8.9 Problema de los óptimos locales

La búsqueda local puede quedar atrapada en mínimos o máximos locales, sin alcanzar el óptimo global.

Estrategias para escapar ★

- Cambiar la estructura de entornos.
- Permitir movimientos a soluciones peores.
- Reiniciar la búsqueda (multiarranque).

8.10 Búsqueda Tabú

Definición Metaheurística basada en búsqueda por entornos con **memoria adaptativa**.

- Permite aceptar **soluciones peores**.
- Evita ciclos mediante una lista tabú.

Tipos de memoria

- Corto plazo: evita repetir movimientos recientes.
- Largo plazo: intensificación y diversificación.

Se pueden crear listas de movimientos tabú que ya han aparecido en soluciones anterior para no realizarse, o también listas de soluciones tabú (aunque tiene menos sentido)

8.11 Recocido simulado

Definición Algoritmo que acepta soluciones peores con cierta probabilidad, consiste en diversificar al principio e intensificar al final.

- Alta diversificación al inicio.
- Intensificación al final.

La probabilidad de aceptar una solución peor depende de:

- La temperatura.
- La diferencia de coste entre soluciones.

8.12 Multiarranque

Proceso que repite:

- Generación de solución inicial.
- Búsqueda local.

hasta cumplir un criterio de parada (basado en número de iteraciones o en evolución de la solución). Se inicia desde varios puntos iniciales aleatorios.

8.13 GRASP

Proceso multiarranque voraz, aleatorio y adaptativo.

- **Voraz**: construye soluciones de buena calidad.
- **Aleatorio**: introduce variabilidad.
- **Adaptativo**: ajusta decisiones según el estado actual.

8.14 Colonia de Hormigas (Ant Colony Optimization - ACO)

8.14.1 Conceptos Clave

- **Naturaleza**: Metaheurística multiagente de búsqueda, que construye probabilísticamente soluciones aprovechando la información de otros agentes.
- **Estigmergia**: Comunicación indirecta a través de rastros de **feromona** en el entorno.
- **Evaporación**: Mecanismo para olvidar malas soluciones y evitar la convergencia prematura (mínimos locales).

8.14.2 Funcionamiento (Ciclo de vida)

1. **Construcción**: Cada hormiga genera una solución paso a paso.
2. **Elección Probabilística**: La hormiga elige el siguiente nodo según:
 - τ (Feromona): Atractivo por experiencia previa del grupo.
 - η (Heurística): Atractivo por cercanía o coste inmediato ($1/distancia$).
3. **Actualización de Feromona**:
 - Se aplica **evaporación** en todos los caminos.

- Se realiza un **depósito** extra en los caminos que forman parte de las mejores soluciones.

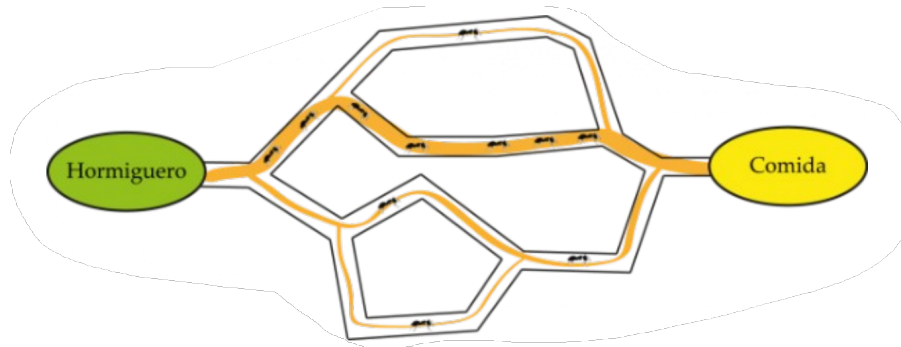


Figura 30: Colonia de hormigas

8.15 Enjambre de Partículas (Particle Swarm Optimization - PSO)

8.15.1 Conceptos Clave

- **Naturaleza:** Metaheurística poblacional para espacios continuos (posición, velocidad).
- **Partícula:** Representa una solución con dos vectores: **Posición** (x) y **Velocidad** (v).
- **Referencias de éxito:**
 - $pbest$: Mejor posición histórica individual (intensificación).
 - $gbest$: Mejor posición histórica de todo el enjambre (diversificación).



Figura 31: Bandada de pájaros

8.15.2 Ecuaciones de Actualización

Actualización de Velocidad

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (pbest_i - x_i(t)) + c_2 r_2 (gbest - x_i(t))$$

- ω : Inercia (mantiene la exploración).

- c_1, c_2 : Coeficientes de aceleración (cognitivo y social), variaremos con más o menos peso a la intensificación o a la diversificación.
- r_1, r_2 : Aleatoriedad uniforme $[0, 1]$.

Actualización de Posición

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

8.16 Hibridación y Otros Métodos

8.16.1 Tipos de Hibridación

- **Serie**: Un algoritmo genera la solución inicial para otro.
- **Paralelo**: Varios algoritmos intercambian mejores soluciones simultáneamente.
- **Integrado**: Una técnica se usa como operador interno de otra (ej: Búsqueda Local dentro de ACO).

9 Tema 7: Algoritmos Genéticos

9.1 Conceptos Fundamentales

- **Naturaleza**: Metaheurística poblacional e iterativa inspirada en la selección natural.
- **Cromosoma (Individuo)**: Representa una solución al problema (generalmente una cadena de bits o números).
- **Población**: Conjunto de individuos que compiten y se reproducen.
- **Fitness (Aptitud)**: Valor numérico que mide qué tan buena es una solución (función objetivo).

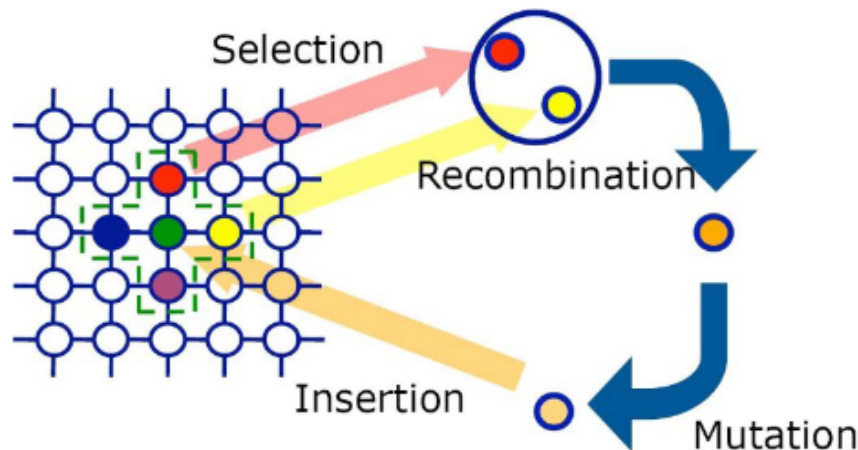


Figura 32: Evolución genética

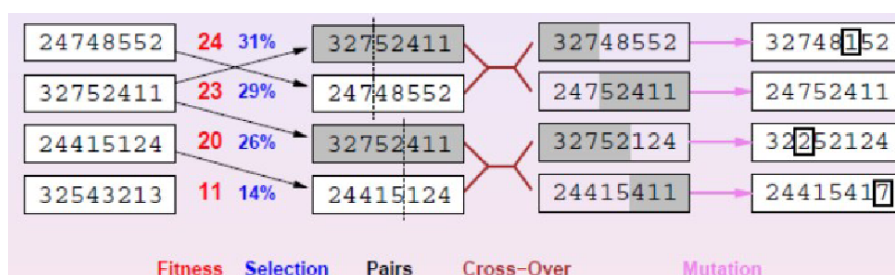


Figura 33: Evolución genética

9.2 Componentes y Operadores Genéticos

9.2.1 1. Representación (Codificación)

Es la forma en que se simboliza una solución (individuo) dentro del algoritmo.

- **Gen:** Unidad mínima de información (parámetro).
- **Cromosoma:** Cadena de genes que representa una solución completa.
- **Tipos comunes:** Binaria (0,1), Real (números flotantes), Permutaciones (orden de elementos) o Árboles.

9.2.2 2. Población Inicial

Conjunto de individuos con el que comienza el algoritmo.

- **Tamaño (N):** Número de individuos. Debe ser suficiente para garantizar diversidad.
- **Generación:** Suele ser aleatoria para cubrir la mayor parte del espacio de búsqueda.

9.2.3 3. Función de Aptitud (Fitness)

Es la función objetivo que evalúa numéricamente qué tan "buena.^{es} una solución.

- Permite comparar individuos y decidir cuáles tienen más probabilidades de sobrevivir y reproducirse.

9.2.4 4. Operadores Genéticos

Mecanismos para generar nuevos individuos (descendientes) a partir de los actuales:

Selección: Elige individuos de la población actual para ser padres, favoreciendo a los de mayor *fitness*.

- **Selección por Ruleta:** La probabilidad de elegir un individuo es proporcional a su *fitness* (a mejor solución, "trozo de ruleta" más grande). Puede causar convergencia prematura si un individuo destaca demasiado.
- **Selección por Torneo:** Se eligen k individuos al azar y el mejor de ellos gana el derecho a ser padre. Permite controlar la **presión selectiva** fácilmente ajustando el tamaño del grupo k .
- **Selección por Rango (Ranking):** Se ordena la población por su *fitness* y se asigna la probabilidad según su posición en la lista. Evita que valores de *fitness* extremadamente altos monopolicen la selección.

Cruce (Crossover): Combina fragmentos de dos cromosomas padres para crear hijos. Es el operador principal de **explotación** (mejora de soluciones).

- **Cruce de 1 punto:** Se elige un punto de corte aleatorio en la cadena. Los genes anteriores al punto se toman del Padre 1 y los posteriores del Padre 2 (y viceversa para el segundo hijo). Es el método más sencillo y clásico.

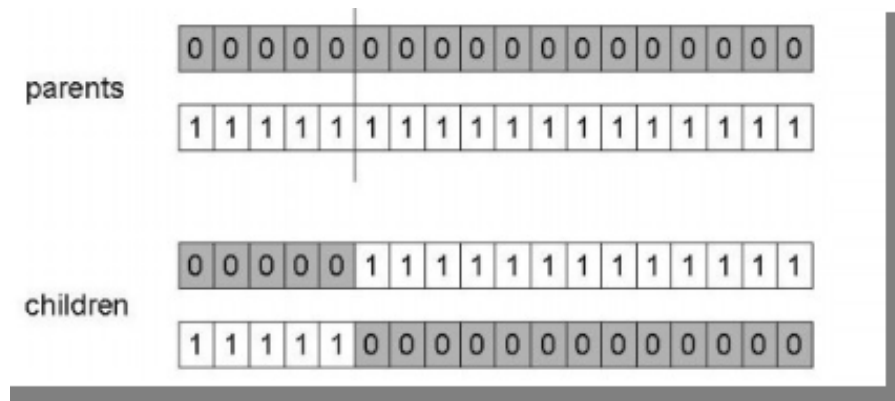


Figura 34: Cruce 1

- **Cruce de n puntos (Multipunto):** Se seleccionan n puntos de corte a lo largo de los cromosomas. Los segmentos de genes se intercambian alternativamente entre los padres. Esto permite una mayor recombinación del material genético y reduce el sesgo derivado del orden de los genes.

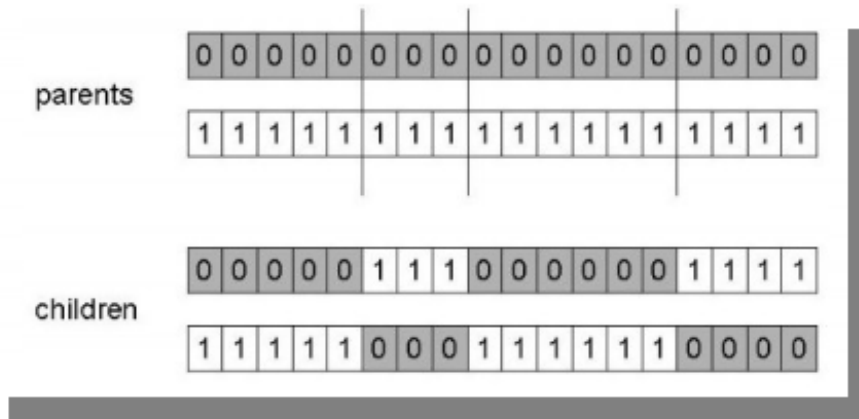


Figura 35: Cruce n

- **Cruce Uniforme:** No utiliza puntos de corte fijos. Para cada gen del hijo, se decide aleatoriamente (normalmente con una probabilidad del 50 %) de cuál de los dos padres se heredará. Proporciona la máxima flexibilidad y diversidad en la mezcla de los genes.

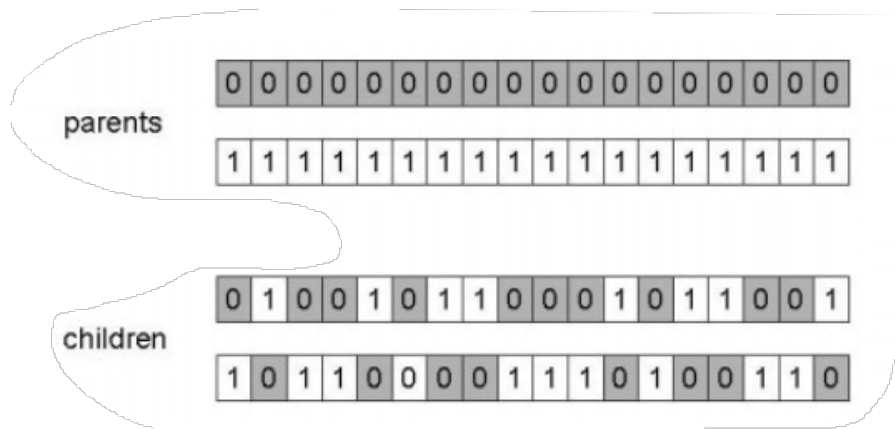


Figura 36: Cruce uniforme

Hay que asegurar la factibilidad, no puede ser por ejemplo en el problema del agente viajero,

que mediante el cruce de padres se obtenga un hijo en el que se repita un nodo. Esto pasa en la mayoría de los problemas

Mutación: Cambia aleatoriamente algún gen. Es el operador de **exploración** (evita mínimos locales y aporta diversidad).

9.2.5 5. Criterio de Parada

Condición que finaliza el algoritmo. Ejemplos:

- Se alcanza un número máximo de generaciones.
- Se encuentra una solución con un *fitness* objetivo.
- La población ha convergido (todos los individuos son muy similares).

9.2.6 4. Mutación

Altera aleatoriamente un gen de un individuo.

- **Función:** Mantener la **diversidad genética** y evitar caer en mínimos locales (exploración).

9.2.7 5. Reemplazo

Decide qué individuos pasan a la siguiente generación.

- **Elitismo:** Asegura que los mejores individuos de la generación actual no se pierdan.

9.3 Esquema General del Algoritmo

1. Generar población inicial aleatoria.
2. Evaluar el *Fitness* de cada individuo.
3. **Mientras** no se cumpla el criterio de parada:
 - **Seleccionar** padres.
 - **Cruzar** padres para obtener hijos.
 - **Mutar** hijos con una probabilidad baja (p_m).
 - **Evaluar** nuevos individuos.
 - **Reemplazar** la población antigua.

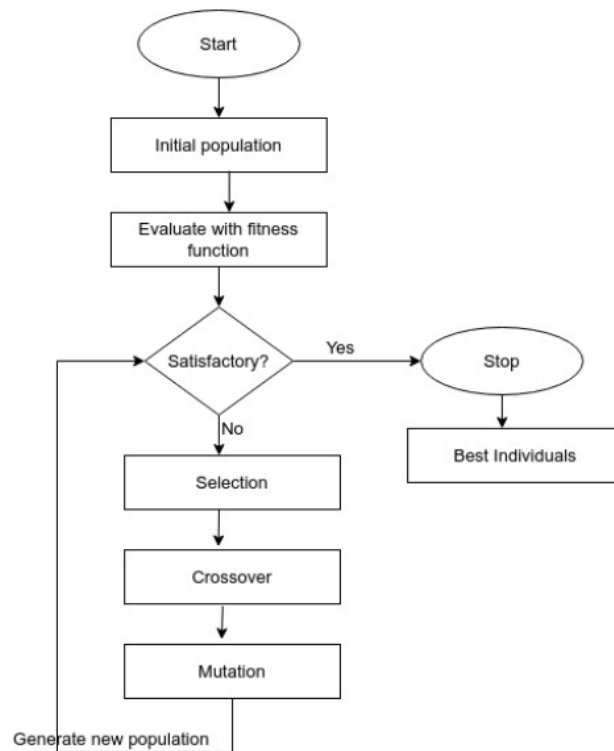


Figura 37: Proceso evolución genética

9.4 Parámetros Críticos

- **Tamaño de la población:** Demasiado pequeña causa convergencia prematura; demasiado grande es lenta.
- **Probabilidad de cruce (p_c):** Suele ser alta (0.6 - 0.9).
- **Probabilidad de mutación (p_m):** Suele ser muy baja (0.001 - 0.05).

9.5 Parámetros

- No necesitan de un conocimiento previo, pero se le puede incorporar.
- Se pueden incorporar con otras técnicas